CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

System Design Document (SDD)

Project Title: CebuNest

Prepared By: Saligue, Kean Maverick

Version: 2

Date: 2/21/2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1	[02/21/2026	Kean Maverick Saligue	(Initial Draft): Added the content for the executive Summary, Introduction, and Functional Requirements Specifications	Final
2	[02/27/2026	[Kean Maverick Saligue	Added contents for System Architecture, API Contract and Communication Flow, Database Design, UI/UX Design, and Project Plan.	Final
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4
1.0 INTRODUCTION 5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5
3.0 NON-FUNCTIONAL REQUIREMENTS 8
4.0 SYSTEM ARCHITECTURE 9
5.0 API CONTRACT & COMMUNICATION 10
Authentication Endpoints 11
User Registration 11
User Login 11
6.0 DATABASE DESIGN 13
7.0 UI/UX DESIGN 14
8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

CebuNest is a simplified rental management platform designed to streamline the renting process for tenants and property owners in Cebu City. The system allows tenants to browse available boarding houses (BH), apartments, and rental units, submit rental requests, and manage their bookings. At the same time, property owners can create and manage property listings, review rental requests, and monitor payments.

The platform follows a three-tier architecture consisting of a **Spring Boot backend API**, a **React web application**, and an **Android mobile application**. These components are integrated through RESTful APIs to provide a seamless, secure, and accessible rental management experience across devices.

1.2 Objectives

1. Develop a fully functional rental management MVP with user authentication, property listings, booking requests, and basic management features.
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile).
3. Create RESTful APIs to enable communication between the backend, web, and mobile applications.
4. Design a clean and responsive user interface that ensures ease of use for both tenants and property owners.
5. Deploy the system in a production-ready setup for real-world use.

1.3 Scope

Included Features:

- User registration and login with JWT authentication
- Role-based access control (Tenant, Owner, and Admin)
- Property listing with search and filter functionality
- Property image upload and file management
- Rental request submission and status management (Pending, Approved, Rejected)
- Owner dashboard for managing property listings and requests
- Sandbox payment gateway integration for fees.
- Protected routes and secured API endpoints
- Responsive web interface (React TypeScript)
- Kotlin Android mobile application

- Sending email using SMTP services
- Google OAuth Login
- Locating property location via OpenStreetMap API

Excluded Features:

- Advanced rent billing automation
- Full-scale property analytics dashboard
- Multi-property enterprise management system

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the **CebuNest** Rental Management System. It outlines the system requirements, architectural design, API contracts, database structure, integrations, and implementation roadmap.

The purpose of this document is to guide the development process and ensure that the backend, web, and mobile components integrate seamlessly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: CebuNest
Domain: Rental Management / Property Listing
Primary Users:

1. Property Owner
2. Tenant

Problem Statement: Tenants often struggle to find available boarding houses and apartments efficiently, while property owners lack a simple system to manage listings and rental requests digitally. Many processes are manual, unorganized, or communicated through social media only.
Solution: CebuNest provides a streamlined rental platform where tenants can browse properties and submit rental requests, and owners can manage listings and approve or reject requests. The system ensures a consistent experience across web and mobile platforms with secure authentication and structured data management.

2.2 Core User Journeys

Journey 1: First-time Tenant Rental Request

1. User visits web or mobile application
2. Clicks "Sign Up" and creates an account and chooses between Tenant and Owner Role
3. Logs in successfully
4. Browses property listings
5. Views detailed property information
6. Submits rental request
7. Receives confirmation notification
8. Receives approval notification from owner
9. Pays reservation fee (sandbox payment)

Journey 2: Returning Tenant

1. User logs in with existing credentials
2. Navigates to "My Rentals" or "Current Bookings" page
3. Views a list of properties they have rented
4. Checks detailed information for each rental, including:

    - Rental period or lease duration
    - Payment status (paid, pending, overdue)
    - Upcoming payment due dates
    - Property owner contact information

5. Makes payment if any installment is due
6. Receives confirmation email and/or in-app notification for completed payment
7. Optionally, views past rental history and receipts

Journey 3: Owner Property Management

1. Owner logs in with Owner credentials
2. Navigates to Owner Dashboard
3. Adds new property with details and image upload
4. Edits property information if needed
5. Views incoming rental requests
6. Approves or rejects requests
7. Verifies payment statuses of properties
8. Attempts to delete a property listing
9. System checks if the property has active or ongoing rentals

- If the property is currently rented or has an active rental request (APPROVED/CONFIRMED), the deletion is rejected
- System displays an error message:
  "This property cannot be deleted because it currently has active tenants."

10. Only able to delete the property after all related rentals are completed or closed

Journey 4: Admin – User Management and Property Approval

1. Admin logs in with Admin credentials
2. Navigates to the Admin Dashboard
3. **Manages Users:**

   - Views all registered users (Tenants, Owners, Admins)
- Creates new user accounts and assigns roles
- Updates user details or roles
- Deactivates or removes users if needed

4. Approves or Rejects Properties

   - Views all property listings submitted by Owners that are pending for reviewing
   - Reviews submitted property details and attached documents (e.g., permits)
   - Approves listings, changing their status to APPROVED
   - Rejects listings, changing their status to REJECTED and optionally providing a reason
   - Notifications are sent to property owners regarding approval or rejection

5. Admin can also view historical actions and audit past approvals/rejections

2.3 Feature List (MoSCoW)

MUST HAVE

1. User Registration
2. User Login
3. Google OAuth Login integration
4. JWT Authentication
5. Logout
6. `/api/auth/me` endpoint
7. Password hashing using BCrypt

8. Protected API routes
9. Role-Based Access Control (Tenant, Owner, and Admin)
10. Property Management (Owner Full CRUD)
11. Property Image Upload (file stored on server + linked to DB)
12. Property Search and Filtering (location, price range)
13. Rental Request Submission (Tenant)
14. Rental Request Approval/Rejection (Owner)
15. Admins can approve or reject property listings before they appear publicly, ensuring only verified properties are available for tenants to request.
16. Rental Request Status Tracking (PENDING, APPROVED, REJECTED)
17. Sandbox Payment Gateway Integration (PayMongo)
18. Payment result recording in database
19. Payment success/failure handling
20. Display property location on interactive map (OpenStreetMap API)
21. SMTP Email Sending (Welcome email + Payment receipt)
22. Role-Based UI (Tenant , Owner, Admin)
23. Property rating and review system

SHOULD HAVE

1. Rental request history page for tenants
2. The option to categorize property listings on owner view.
3. Owner dashboard analytics (basic counts of properties and requests)
4. Property availability indicator
5. Image preview before upload
6. Input validation feedback with specific error messages
7. Rate limiting for login attempts
8. Refresh token implementation
9. Real-time request updates via polling
10. Tenant profile update feature

COULD HAVE

1. WebSocket-based real-time rental updates
2. SMS notification integration
3. Admin role for monitoring all activity
4. AI Chatbot
5. In-app messaging/chat system

WON'T HAVE

1. Automated monthly rent billing system
2. Multi-language support
3. Advanced analytics dashboard
4. Microservices architecture (system remains monolithic layered architecture)

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login
- **Fields:** Full Name, Email, Password, Confirm Password, Role (Tenant or Owner)
- **Validation:** Valid email format and Strong password.
- **API Endpoints:**
  - POST /api/auth/register
  - POST /api/auth/login
  - POST /api/auth/logout
  - GET /api/auth/me
- **Security:**
  - JWT authentication
  - BCrypt password hashing
  - Protected routes

Feature: Property Management

- **Screens:** Property Listing, Property Detail, and Owner Dashboard.
- **Display**:

  - Grid layout
  - Property images (optional)
  - Title, price, location

- **Search & Filter:** By location and price range
- **API Endpoints:**
  - GET /api/properties
  - GET /api/properties/{id}
  - GET /api/properties/search
  - POST /api/properties (Owner only)
  - PUT /api/properties/{id} (Owner only)
  - DELETE /api/properties/{id} (Owner only)

Feature: Rental Request (Tenant)

- **Screens:** Rental Request Form and Request Status Page
- **Functions:**
  - Submit request (Tenant)
  - Approve/reject requests (Owner)
  - Track request status

    - Automatically send email notification upon approval or rejection using an integrated **SMTP-based email service**
    - **Persistence:** Stored in database linked to tenant and property
- **API Endpoints:**
    - POST /api/rental-requests
    - GET /api/rental-requests/my
    - PUT /api/rental-requests/{id}/status

Feature: Payment Integration (Sandbox)

- **Screens:** Payment Page, Payment Result Page
- **Process:**

    - Redirect to payment provider (test mode)
    - Record payment result in database
    - Update rental request status
    - Automatically send email notification upon approval or rejection using an integrated SMTP-based email service

API Endpoints:

- POST /api/payments/create
- POST /api/payments/webhook

Feature: Admin Rental Property Approval

Screens: Admin Dashboard → Pending Property Requests

- **Functions:**
    - Review rental property requests submitted by Owners
    - Approve or reject requests based on submitted details and documents
    - Notify Owner of approval or rejection with reason
- **Persistence:** Property request status stored in database (PENDING_REVIEW, APPROVED, REJECTED)
- **API Endpoints:**
    - GET /admin/rental-requests/pending – Fetch all pending property requests
    - PUT /admin/rental-requests/{id}/status – Update request status (APPROVED | REJECTED)
    - PUT /api/rental-requests/{id}/status

2.5 Acceptance Criteria

AC-1: Successful User Registration

➢Given I am a new user
➢When I enter a valid email and strong password
➢And confirm that the passwords match
➢And click "Create Account"
➢Then my account should be created
➢And my password should be securely hashed
➢And redirected to the Login Page

AC-2: Tenant Rental Request Flow

➢ Given I am logged in as a tenant
➢ When I view an available property
➢ And click "Request to Rent"
➢ And enter a valid start date and lease duration
➢ And submit the rental request
➢ Then the rental request should be created
➢ And the rental status should be set to **PENDING**
➢ And the property owner should be notified
➢ And I should receive an automated email notification and app notification once the owner approves or rejects the request

---

AC-3: Owner Property Management

➢ Given I am logged in as a property owner
➢When I view a rental request with status PENDING
➢And choose to approve the request
➢Then the rental status should change to APPROVED
➢And the tenant should be notified via email and app notification
➢And the payment option should become available

➢And When I choose to reject the request
➢Then the rental status should change to REJECTED

➢And the tenant should be notified via email and app notification
➢And payment should not be allowed

➢ And when a property belongs to a specific category
➢ Then the Owner should be able to create targeted in-app notifications
➢ And those notifications should be visible only to tenants who are currently renting properties within that category

---

AC-4: Rental Payment and Confirmation

➢Given I am logged in as a tenant
➢And my rental request status is APPROVED
➢And when I complete the reservation payment through the payment gateway
➢Then the payment should be validated
➢And a payment record should be stored
➢And the rental status should change to CONFIRMED
➢And the property should be marked as unavailable
➢And the owner should receive payment confirmation via email and app notification

---

AC-5: Admin Property Listing Approval

➢ Given I am logged in as an Admin
➢ When a property owner submits a new property listing
➢ And the property includes details and required documents (e.g., business permit or verification documents)
➢ Then the property listing should be marked as PENDING_REVIEW
➢ And the Admin should be able to review the submitted listing
➢ And When the Admin approves the listing
➢ Then the property status should change to APPROVED
➢ And the property becomes visible to tenants in the property listings
➢ And the owner should receive an email and notification of approval
➢ And When the Admin rejects the listing
➢ Then the property status should change to REJECTED
➢ And the owner should receive an email and notification explaining the reason for rejection (optional).
➢And the Admin should be able to create and send in-app notifications to users (Tenants or Owners) for announcements, updates, or system-related information

AC-6: Admin User Management

➤ Given I am logged in as an Admin
➤ When I access the Admin Dashboard
➤ Then I should be able to view all registered users in the system

➤ And When I create a new user account
➤ Then the system should store the user information securely
➤ And assign the selected role (Tenant, Owner, or Admin)

➤ And When I update a user's role or information
➤ Then the system should save the updated details successfully

➤ And When I deactivate a user account
➤ Then the user should no longer be able to access the system

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- **Average response time:** ≤ 2 seconds for 95% of API requests
- **Maximum response time:** ≤ 5 seconds under high load
- **Web page load time:** ≤ 3 seconds on broadband
- **Mobile app cold start:** ≤ 3 seconds
- **Concurrent users supported:** ≥ 50 simultaneous users
- **Database query execution:** ≤ 500ms for standard queries

3.2 Security Requirements

- **Data transport:** HTTPS for all communications
- **Authentication:** JWT token authentication for all protected endpoints
- **Password storage:** bcrypt hashing (salt rounds = 12)
- **Vulnerability protection:** SQL injection prevention on all input fields
- **Access control:** Role-based access for tenants and owners; admin endpoints restricted

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)

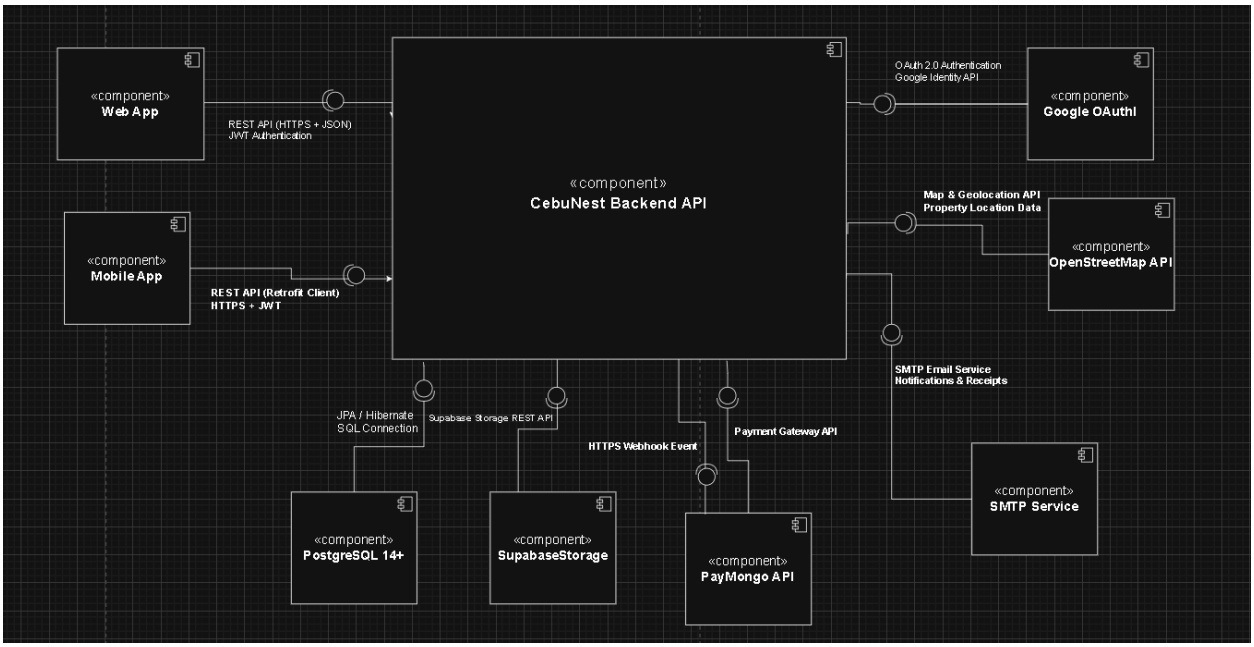
- **Android:** API Level 34 (Android 14)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+
- **Backend API:** Compatible with REST clients and mobile Retrofit library

3.4 Usability Requirements

- **Tenant onboarding:** Complete account registration and first rental request within 5 minutes
- **Accessibility:** WCAG 2.1 Level AA compliance for web interfaces
- **Navigation:** Consistent menu and dashboard experience across web and mobile
- **Error handling:** Clear, actionable messages with recovery options
- **Mobile interaction:** Touch targets minimum 44x44px; supports swipe gestures for property browsing

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.5.11, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+ (SupaBase)
- **Web Frontend:** React 18, TypeScript
- **Mobile:** Kotlin and Retrofit
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://api.cebunest.com/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	{
	"success": boolean,
	"data": object|null,
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

Description	Register a new user account
API URL	/auth/register
HTTP Request Method	POST
Format	JSON
Authentication	None

Request Payload	{
	"email": "<email>",
	"password": "<password>",
	"confirmPassword": "<confirmPassword>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "TENANT | OWNER"
	}

Response Structure	{
	"success": true,
	"data": {
	"user": {
	"id": "<id>",
	"email": "<email>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "<role>"

	}, "accessToken": "<token>", "refreshToken": "<token>" }, "error": null, "timestamp": "<timestamp>" }

Description	User Login
Format	/auth/login
HTTP Request Method	POST
Format	JSON
Authentication	None

Request Payload	{
	"email": "<email>",
	"password": "<password>"
	}

Response Structure	{
	"success": true,
	"data": {
	"user": {
	"id": "<id>",
	"email": "<email>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "<role>"
	},
	"accessToken": "<token>",
	"refreshToken": "<token>"
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Logout authenticated user
API URL	/auth/logout
HTTP Request Method	POST
Format	JSON
Authentication	Bearer Token Required
Header	Authorization: Bearer <token>

Response Structure	{
	"success": true,
	"data": {
	"message": "Logged out successfully"
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Get currently logged-in user
API URL	/auth/me
HTTP Request Method	GET
Format	JSON
Authentication	Bearer Token Required
Header	Authorization: Bearer <token>

Response Structure	{
	"success": true,
	"data": {
	"user": {
	"id": "<id>",
	"email": "<email>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "<role>"
	}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Property Endpoints

Description	Retrieve List of properties with filters
API URL	/properties
HTTP Request Method	GET
Format	JSON
Authentication	Tenant
Query Parameters	?page=1 &limit=20 &search=location &minPrice=0 &maxPrice=10000

Response Structure	{
	"success": true,
	"data": {
	"properties": [],
	"pagination": {
	"page": 1,
	"limit": 20,
	"total": 100,
	"pages": 5
	}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Retrieve specific property
API URL	/properties/{id}
HTTP Request Method	GET
Format	JSON
Authentication	Bearer Token Required

Response Structure	{
	"success": true,
	"data": {
	"property": {
	"id": "<id>",
	"title": "<title>",
	"description": "<description>",
	"price": "<price>",
	"location": "<location>",
	"ownerId": "<ownerId>",
	"images": [],
	"status": "<status>"
	}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Create a new property listing
API URL	/properties
HTTP Request Method	POST
Format	JSON

Authentication	Owner Only

Request Payload	{
	"title": "<title>",
	"description": "<description>",
	"price": "<price>",
	"location": "<location>",
	"images": []
	}

Response Structure	{
	"success": true,
	"data": {
	"property": {}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Update property details
API URL	/properties/{id}
HTTP Request Method	PUT
Format	JSON
Authentication	Owner Only

Request Payload	{
	"title": "<optional>",
	"description": "<optional>",
	"price": "<optional>",
	"location": "<optional>",
	"images": []
	}

Response Structure	{
	"success": true,
	"data": {
	"property": {}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Delete property
API URL	/properties/{id}
HTTP Request Method	DELETE
Format	JSON
Authentication	Owner

Response Structure	{
	"success": false,
	"data": null,
	"error": {
	"code": "BUSINESS-003",
	"message": "Property has active tenants",
	"details": "Cannot delete property with active rentals"
	},
	"timestamp": "<timestamp>"
	}

Description	Tenant submits a review for a property
API URL	/property-reviews
HTTP Request Method	POST
Format	JSON
Authentication	Tenant

Request Payload	{
	"propertyId": "<propertyId>",
	"rating": 1-5,
	"comment": "<comment>"
	}

Response Structure	{
	"success": true,
	"data": {
	"propertyReview": {
	"id": "<reviewId>",
	"propertyId": "<propertyId>",
	"tenantId": "<tenantId>",
	"rating": "<rating>",
	"comment": "<comment>",
	"createdAt": "<timestamp>"
	}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Retrieve all reviews for a specific property
API URL	/property-reviews/property/{propertyId}
HTTP Request Method	GET
Format	JSON
Authentication	Optional

Response Structure	{
	"success": true,
	"data": {
	"propertyReviews": [
	{
	"id": "<reviewId>",
	"propertyId": "<propertyId>",
	"tenantId": "<tenantId>",
	"rating": "<rating>",
	"comment": "<comment>",
	"createdAt": "<timestamp>"
	}
	]
	},
	"error": null,
	"timestamp": "<timestamp>" }

Rental Request Endpoints

Description	Tenant submits rental request
API URL	/rental-requests
HTTP Request Method	POST
Format	JSON
Authentication	Tenant

Response Structure	{
	"success": true,
	"data": {
	"rentalRequest": {
	"id": "<id>",
	"propertyId": "<propertyId>",
	"tenantId": "<tenantId>",
	"status": "PENDING",
	"createdAt": "<timestamp>"
	}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}}

Description	Retrieve tenant rental requests
API URL	/rental-requests/my
HTTP Request Method	GET
Format	JSON
Authentication	Tenant

Response Structure	{
	"success": true,
	"data": {
	"rentalRequests": []
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Owner views rental requests for property
API URL	/rental-requests/property/{propertyId}
HTTP Request Method	GET
Format	JSON
Authentication	Owner Only

Response Structure	{
	"success": true,
	"data": {
	"rentalRequests": []
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Owner approves or rejects rental requests
API URL	/rental-requests/{id}/status
HTTP Request Method	PUT
Format	JSON
Authentication	Owner Only

Request Payload	{
	"status": "APPROVED | REJECTED"
	}

Response Structure	{
	"success": true,
	"data": {
	"rentalRequest": {}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Payment Endpoints

Description	Create payment for rental
API URL	/payments/create
HTTP Request Method	POST
Format	JSON
Authentication	Tenant Only

Request Payload	{
	"rentalRequestId": "<id>",
	"amount": "<amount>"
	}

Response Structure	{
	"success": true,
	"data": {
	"payment": {
	"id": "<id>",
	"status": "<status>",
	"paymentUrl": "<url>"
	}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Payment provider webhook
API URL	/payments/webhook
HTTP Request Method	POST
Format	JSON
Authentication	None

Request Payload	{
	"paymentEventData": {}
	}

Response Structure	{
	"success": true,
	"data": {
	"message": "Webhook received"
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Retrieve payment details
API URL	/payments/{id}
HTTP Request Method	GET
Format	JSON
Authentication	Tenant / Owner

Response Structure	{
	"success": true,
	"data": {
	"payment": {
	"id": "<id>",
	"rentalRequestId": "<id>",
	"amount": "<amount>",
	"status": "<PAID | FAILED | PENDING>",
	"provider": "PayMongo",
	"createdAt": "<timestamp>"
	}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Notification Endpoints

Description	Retrieve all notifications for a specific tenant (Owner or admin generated)
API URL	/notifications/tenant/{tenantId}/{notificationId}/read
HTTP Request Method	PUT
Format	JSON
Authentication	Role-based

Response Structure	{
	"success": true,
	"data": {
	"message": "Notification marked as read",
	"notification": {
	"id": "<notificationId>",
	"tenantId": "<tenantId>",
	"read": true
	}
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

Description	Retrieve all notifications for a specific property owner (e.g., rental requests, payments)
API URL	/notifications/owner/{id}
HTTP Request Method	GET
Format	JSON
Authentication	Role-based

Response Structure	{
	"success": true,
	"data": {
	"notifications": [
	{
	"id": "<notificationId>",
	"title": "<Notification Title>",
	"message": "<Notification Message>",
	"type": "<INFO | ALERT | RENTAL_REQUEST | PAYMENT>",
	"ownerId": "<ownerId>",
	"createdAt": "<timestamp>",
	"read": false
	}
	]
	},
	"error": null,
	"timestamp": "<timestamp>"
	}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	json
	{
	"success": false,
	"data": null,
	"error": {
	"code": "BUSINESS-002",
	"message": "Property unavailable",
	"details": "This property is no longer available for new rental requests."
	},
	"timestamp": "2026-02-21T10:30:00Z"
	}

Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"startDate": "Start date is required",
	"leaseDurationMonths": "Lease duration must be at least 1 month"
	}
	},
	"timestamp": "2026-02-21T10:30:00Z"
	}

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry

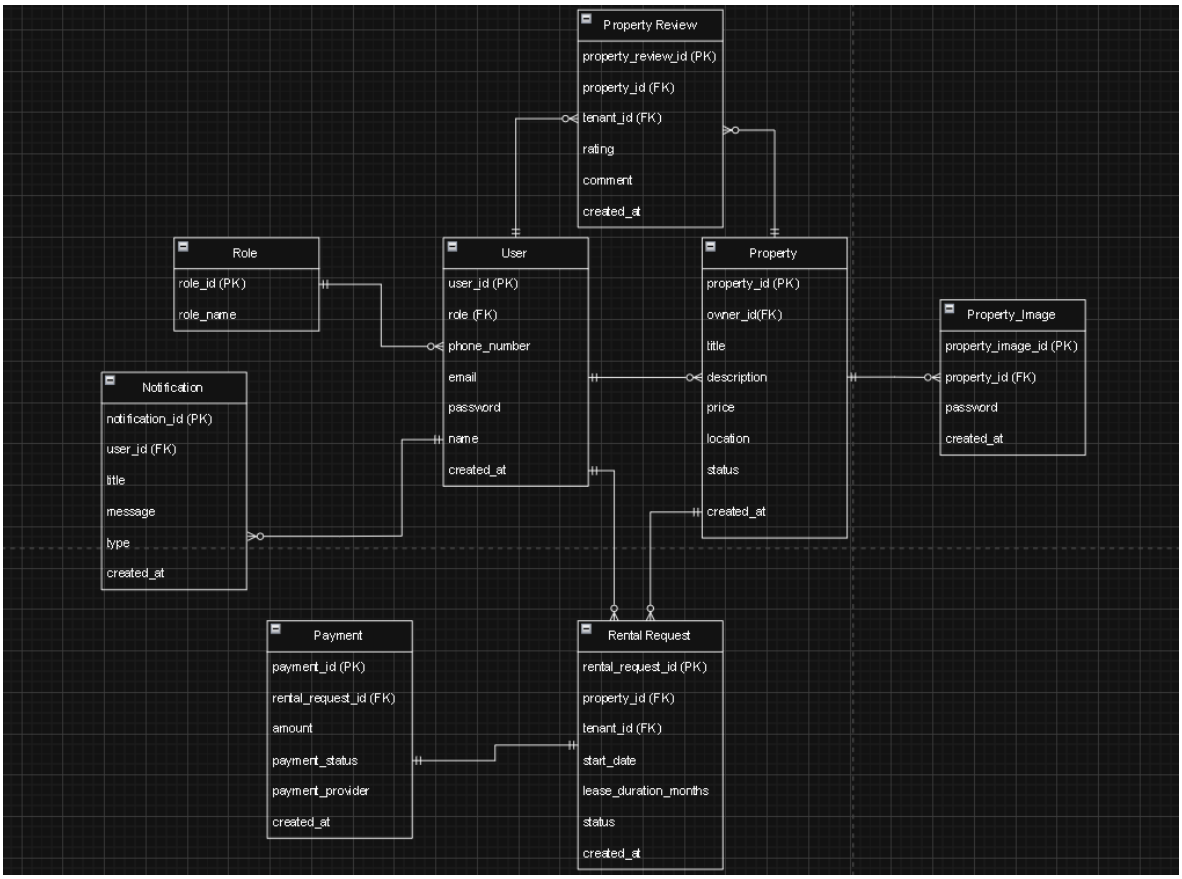
• BUSINESS-001: Rental request cannot be processed
• BUSINESS-002: Property is no longer available

• PAYMENT-001: Payment failed
• PAYMENT-002: Payment verification failed

• SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- One-to-Many: Role → Users (Each role can be assigned to multiple users)
- One-to-Many: User (Owner) → Properties (An owner can create multiple property listings)
- One-to-Many: Property → PropertyImages (Each property can have multiple images)
- One-to-Many: User → Notifications (A user can receive multiple notifications)
- One-to-Many: Property → RentalRequests (A property can receive multiple rental requests)

- One-to-Many: User (Tenant) → RentalRequests (A tenant can submit multiple rental requests)
- One-to-One: RentalRequest → Payment (Each approved rental request results in one payment record)
- One-to-Many: Property → PropertyReviews (A property can have multiple reviews from tenants)
- One-to-Many: User (Tenant) → PropertyReviews (A tenant can write multiple property reviews)

Key Tables:

1. roles - Defines system roles such as Tenant, Owner, and Admin
2. users - User accounts and authentication
3. properties - Property listings created by owners
4. property_images - Images associated with properties
5. rental_requests - Rental applications submitted by tenants
6. payments - Reservation payments for approved rental requests
7. notifications - System notifications sent to users
8. property_reviews - Reviews and ratings given by tenants

Table Structure Summary:

● **roles:** role_id, role_name

● **users:** user_id, role_id, name, email, phone_number, password, created_at

● **properties:** property_id, owner_id, title, description, price, location, status, created_at

● **property_images:** property_image_id, property_id, image_url, created_at

● **rental_requests:** rental_request_id, property_id, tenant_id, start_date, lease_duration_months, status, created_at

● **payments:** payment_id, rental_request_id, amount, payment_status, payment_provider, created_at

● **notifications:** notification_id, user_id, title, message, type, created_at

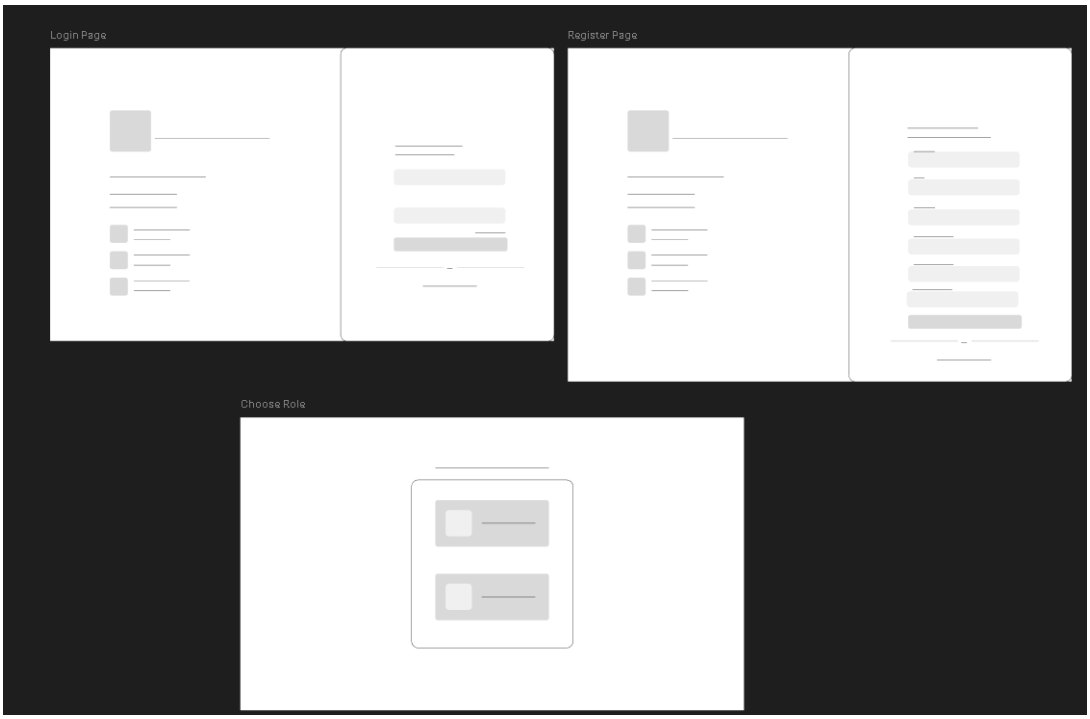
● **property_reviews:** property_review_id, property_id, tenant_id, rating, comment, created_at

7.0 UI/UX DESIGN

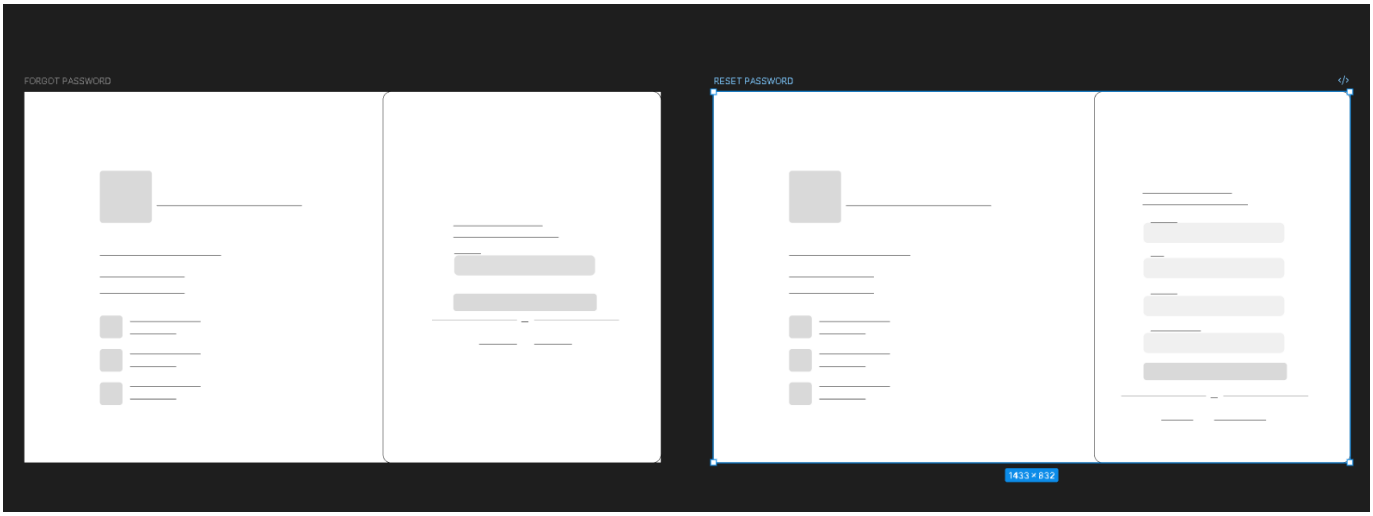
7.1 Web Application Wireframes

The mobile version of CebuNest will not include admin screens; these will be available only on the web version.

Login and Register Page

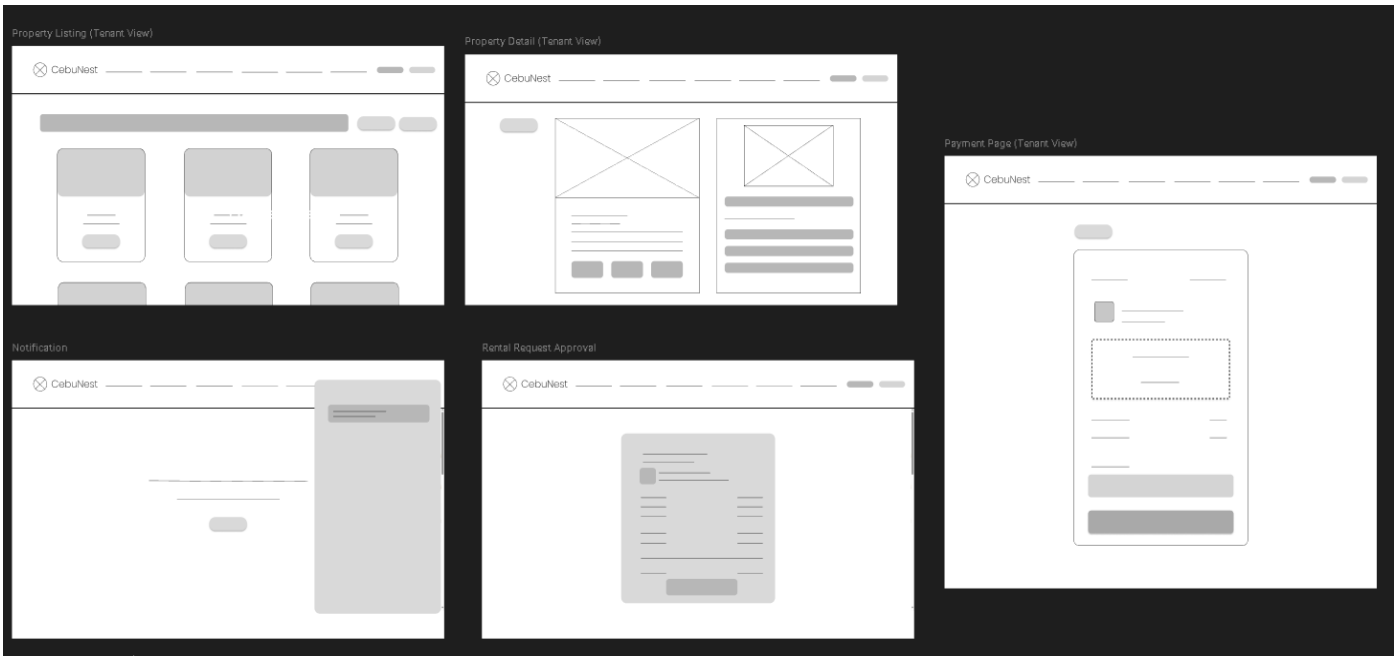


Forgot Password Page



Landing Page (Tenant)

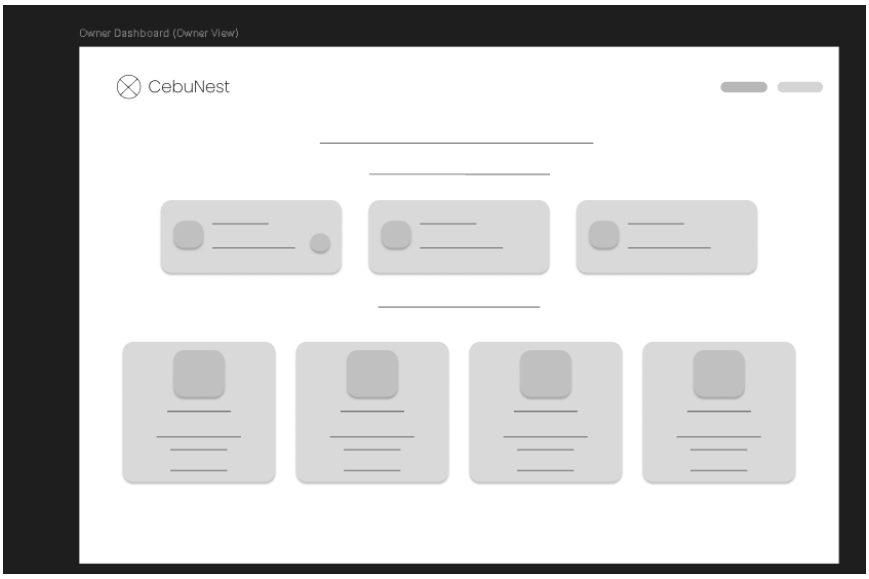
Renting Property Pages Flow (Tenant View)



Profile Page (Tenant)

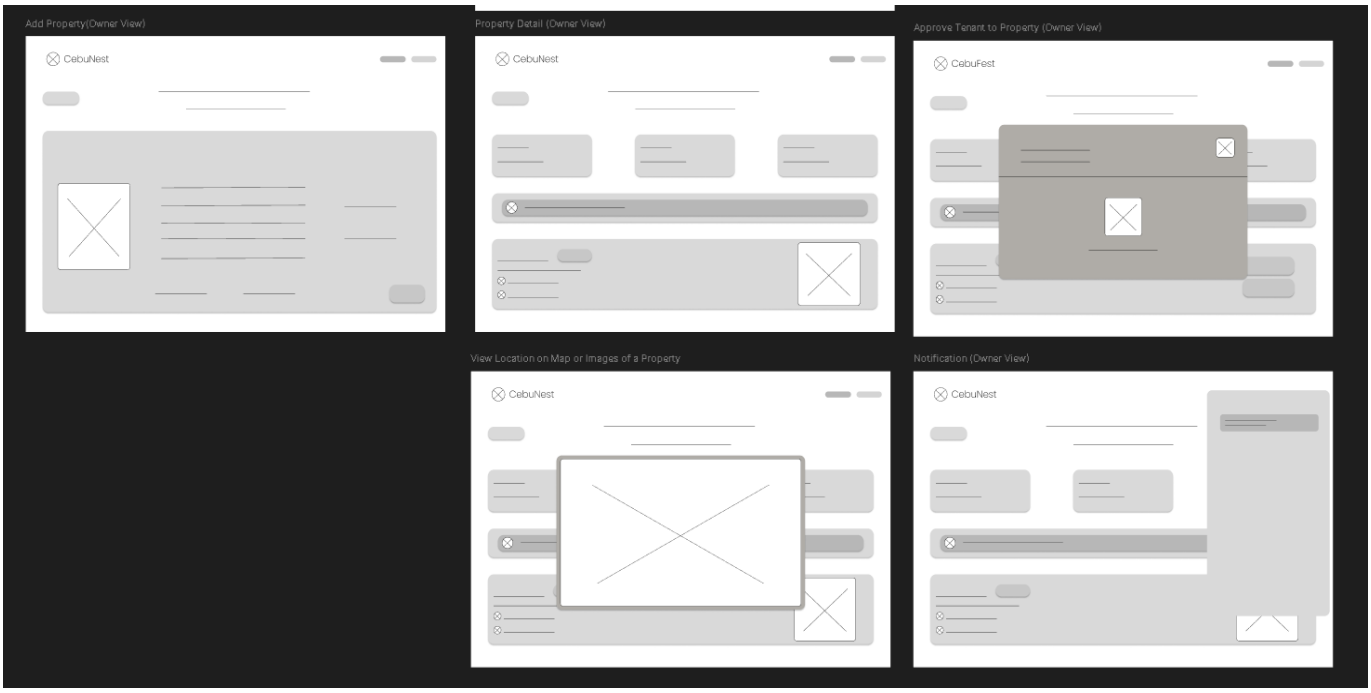
Rented Property Detail Page (Tenant)

Owner Dashboard (Owner)

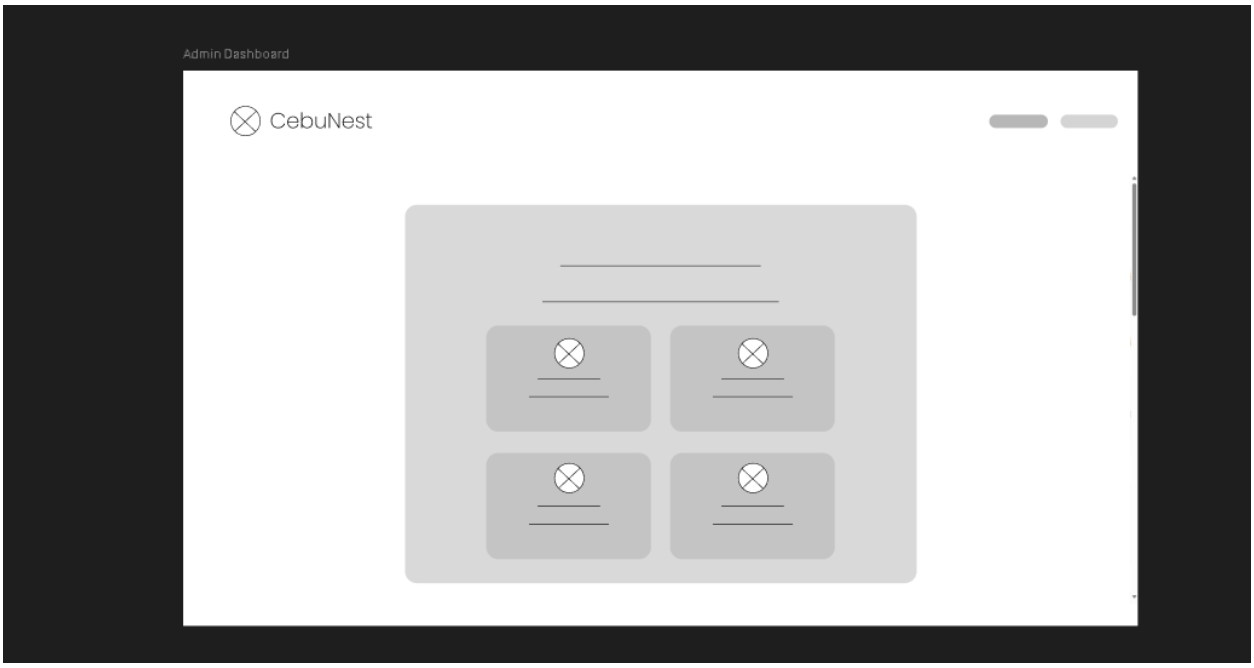


Owner Property List (Owner)

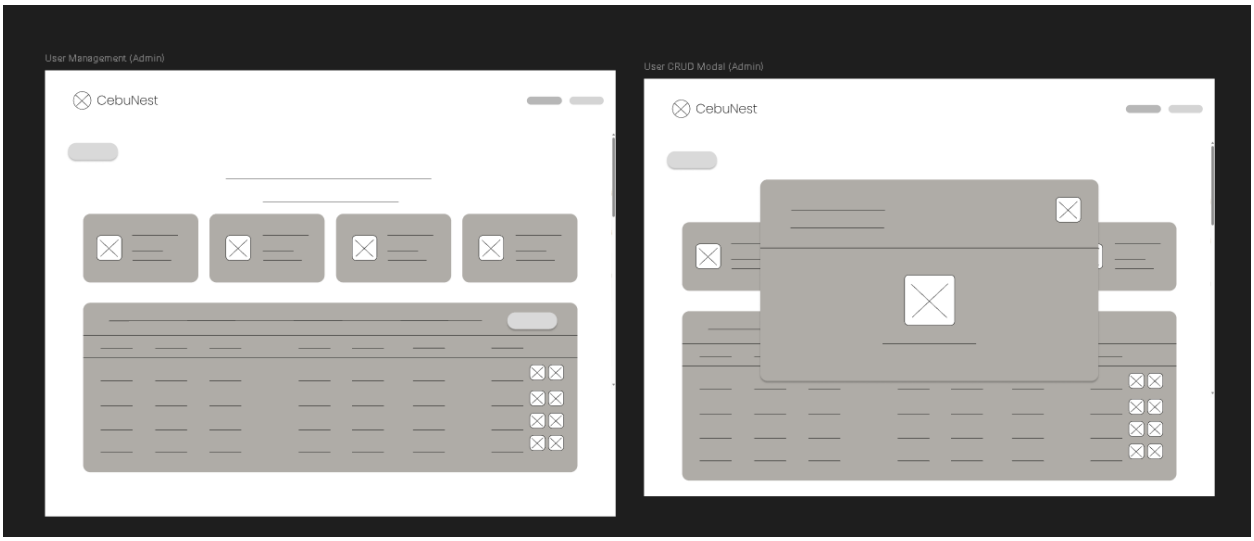
Property Pages Flow (Owner)



Admin Dashboard (Admin)



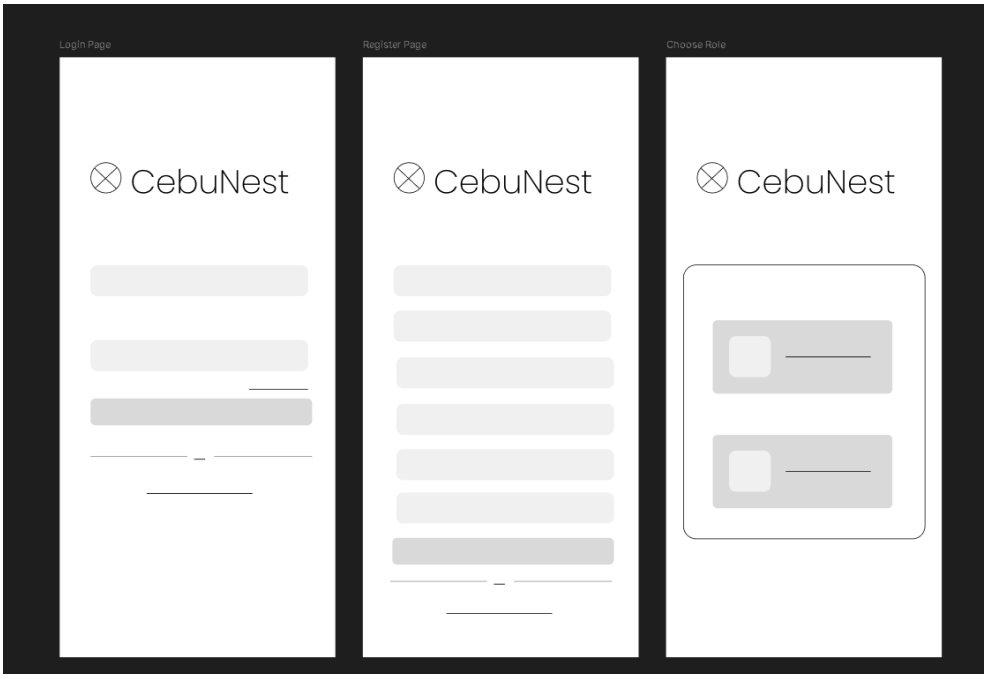
Admin User Management Flow (Admin)



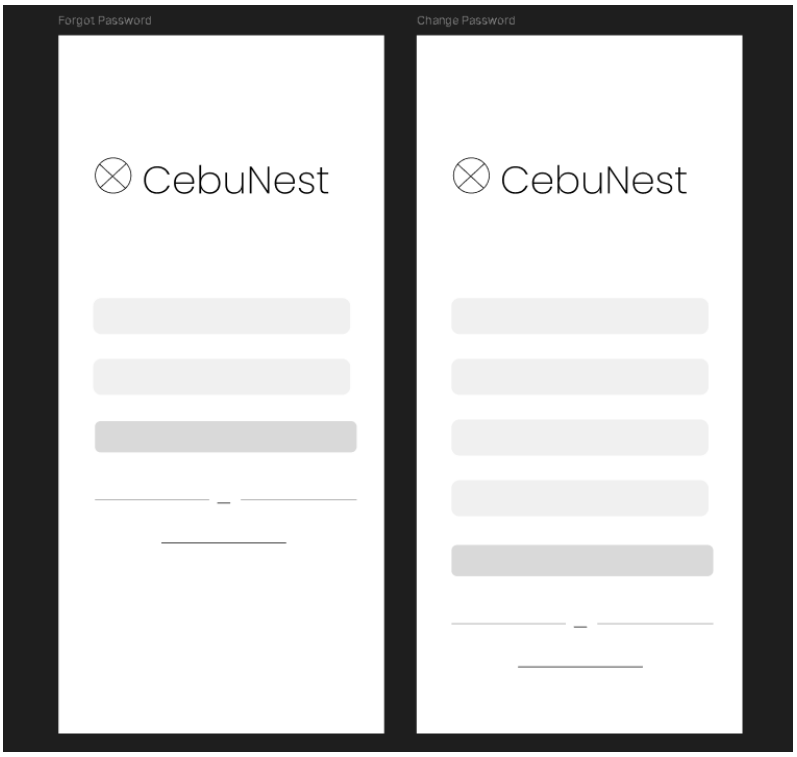
Admin Property Approval Management Flow (Admin)

7.2 Mobile Application Wireframes

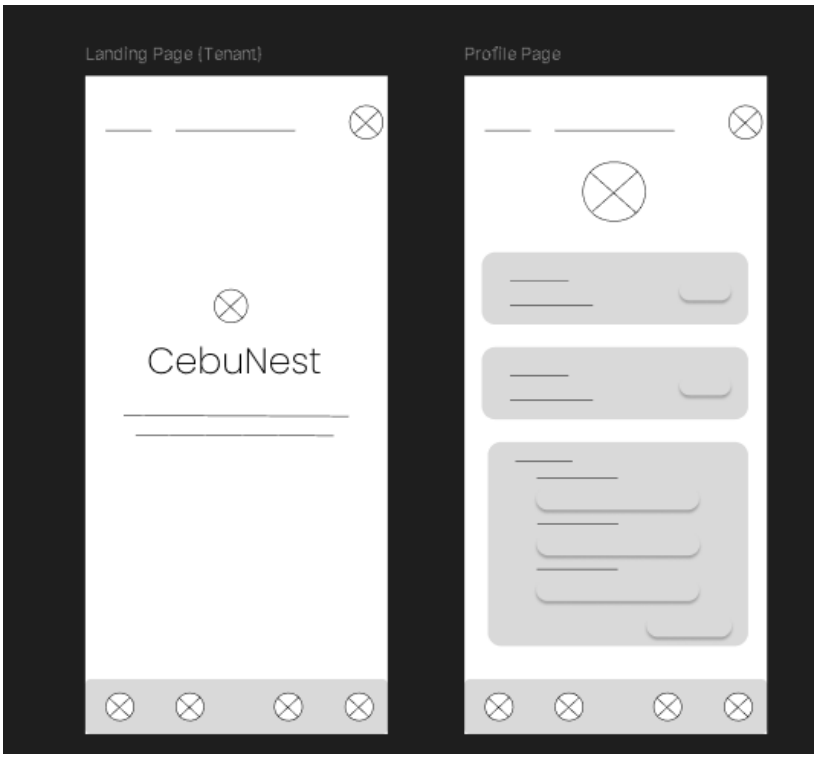
Login Page and Register Page



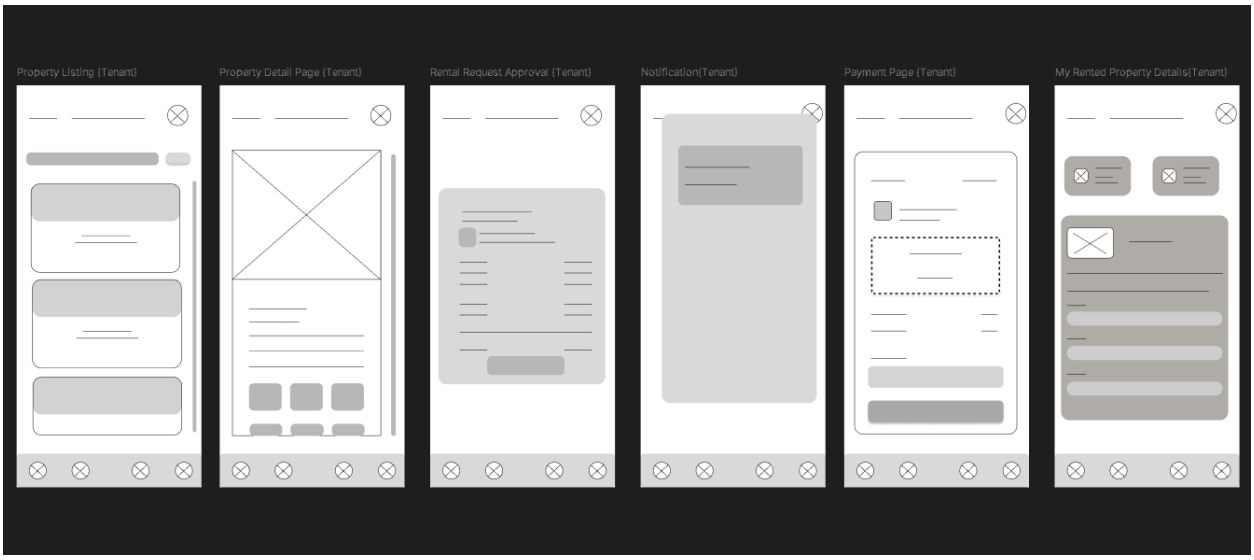
Forgot Password and Change Password Page



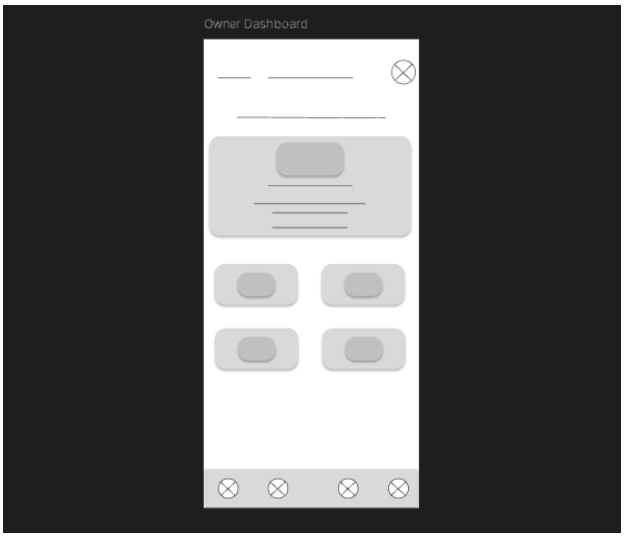
Landing Page and Profile Page (Tenant)



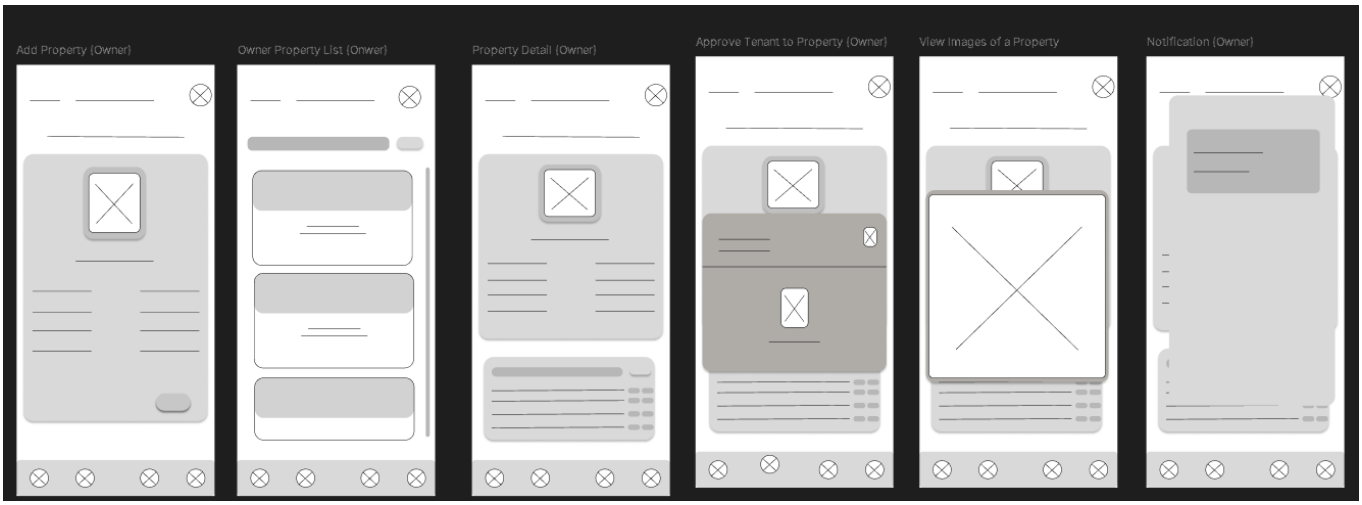
Renting Property Workflow Pages(Tenant)



Owner Dashboard Page



Property Workflow Pages



8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1–2: Project setup and documentation
Day 3–4: Complete Functional and Non-Functional Requirements
Day 5–7: System architecture design and technology stack confirmation

Week 2: Detailed Design

Day 1–2: API specification and endpoint design
Day 3–4: Database schema and ERD finalization
Day 5–6: UI/UX wireframes in Figma (Tenant, Owner, Admin)
Day 7: Final review of design documents

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot project setup and dependencies
Day 2: Database configuration (PostgreSQL / Supabase) and entity models
Day 3: JWT authentication and role-based access control
Day 4: User management APIs (Tenant, Owner, Admin)
Day 5: Property management APIs (Create, Update, Delete, View)

Week 4: Core Features

Day 1: Rental request system implementation
Day 2: Property approval workflow (Admin review system)
Day 3: Payment integration setup (PayMongo sandbox)
Day 4: Email notification system (SMTP integration)
Day 5: API testing, validation, and documentation

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React + TypeScript project setup
Day 2: Authentication pages (Login, Register, OAuth login)
Day 3: Property listing page with search and filters
Day 4: Property details page
Day 5: Rental request submission interface

Week 6: Complete Web Features

Day 1: Tenant dashboard (My Rentals, payment status tracking)
Day 2: Owner dashboard (Property management and request handling)
Day 3: Admin dashboard (User management and property approval)
Day 4: Responsive UI improvements and usability testing
Day 5: Frontend integration with backend APIs

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project configuration
Day 2: Authentication screens implementation
Day 3: Property browsing and search features
Day 4: Rental request functionality
Day 5: API service integration using Retrofit

Week 8: Complete Mobile App

Day 1: Tenant rental management and payment status view
Day 2: Owner property management features
Day 3: UI polish and performance improvements
Day 4: Testing on emulator and physical devices
Day 5: APK build generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across backend, web, and mobile
Day 2: Bug fixing and performance optimization
Day 3: Security review and authentication testing
Day 4: Database and API performance testing
Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway / Render / Heroku)
Day 2: Web application deployment (Vercel / Netlify)
Day 3: Mobile APK distribution for testing
Day 4: Final system validation
Day 5: Project submission and presentation preparation

Milestones:

- M1 (End of Week 2): System design and documentation completed
- M2 (End of Week 4): Backend API fully implemented
- M3 (End of Week 6): Web application completed
- M4 (End of Week 8): Mobile application completed
- M5 (End of Week 10): Fully integrated and deployed system

Critical Path:

1. Authentication and role-based access control implementation (Week 3)

2. Property listing and management system (Week 3–4)
3. Rental request workflow (Week 4)
4. Admin property approval system (Week 4)
5. Payment integration and confirmation flow (Week 4–6)
6. Cross-platform system testing (Week 9)

Risk Mitigation:

- Develop and test core features first (authentication, property listings, rental requests)
- Perform integration testing early to detect API issues
- Maintain version control backups using GitHub
- Monitor API performance and database queries
- Focus on MVP functionality before adding enhancements